

PARAMETERIZED ALU DESIGN AND VERIFICATION REPORT

Author Name: T G Sunag

Employee ID: 6941

Project Title: Parameterized ALU using Verilog HDL

Date: May 2026

1. PROJECT INTRODUCTION

The Parameterized Arithmetic Logic Unit (ALU) is a flexible digital design implemented in Verilog HDL to perform a variety of arithmetic and logical operations. By utilizing parameters, the architecture allows for seamless scaling of data bit-widths, making it a highly reusable component for diverse processor configurations. This project focuses on high-reliability hardware design, employing a self-checking testbench architecture to automate the verification process. This testbench integrates a Reference Model—an algorithmic golden model—that runs in parallel with the RTL, comparing results in real-time to detect functional discrepancies. Through this rigorous approach, the design ensures mathematical accuracy and flag generation (such as Carry, Zero, and Overflow) across all parameterized widths. This report details the modular RTL implementation, the logic behind the automated checking mechanism, and the simulation results that validate the ALU's performance.

2. OBJECTIVES

- **Scalability:** Enable dynamic data width adjustment using Verilog parameters without code changes.
- **Functionality:** Support a standard set of arithmetic and bitwise instructions via opcode selection.
- **Flag Logic:** Implement status outputs for Carry, Zero, Negative, and Overflow conditions.
- **Automation:** Build a self-checking environment to eliminate manual waveform inspection.
- **Accuracy:** Utilize a behavioural Reference Model to validate every calculation cycle.

3. DESIGN ARCHITECTURE

3.1 Architecture Overview

The ALU is an RTL module that selects operations based on MODE and CMD control signals. Key Internal Units include:

- Arithmetic Unit: Handles addition, subtraction, and multiplication.
- Logical Unit: Executes bitwise AND, OR, XOR, etc.
- Comparator Unit: Determines Greater, Less, or Equal status.
- Shift/Rotate Unit: Manages bit manipulations.
- Control Decoder: Interprets input commands to select the active unit.

3.2 Input Signals

Signal	Width	Description
OPA	Parameterized	Operand A input
OPB	Parameterized	Operand B input
CIN	1-bit	Carry input signal
CLK	1-bit	Clock signal
RST	1-bit	Active high reset
CE	1-bit	Clock enable signal
MODE	1-bit	Selects arithmetic or logical mode
INP_VALID	2-bit	Indicates valid operands
CMD	4-bit	Operation select command

3.3 Output Signals

Signal	Width	Description
RES	2 × WIDTH	ALU operation result
OFLOW	1-bit	Overflow indication
COUT	1-bit	Carry/Borrow output
G / L / E	1-bit	Greater, Less, and Equal comparator flags
ERR	1-bit	Error indication signal

4. SUPPORTED OPERATIONS

Logical Operations (MODE = 0)

- CMD 0: AND
- CMD 1: NAND
- CMD 2: OR
- CMD 3: NOR
- CMD 4: XOR
- CMD 5: XNOR
- CMD 6: NOT Operand A (NOT_A)
- CMD 7: NOT Operand B (NOT_B)
- CMD 8: Right Shift Operand A (OPA >> 1)
- CMD 9: Left Shift Operand A (OPA << 1)
- CMD 10: Right Shift Operand B (OPB >> 1)
- CMD 11: Left Shift Operand B (OPB << 1)
- CMD 12: Rotate Left
- CMD 13: Rotate Right

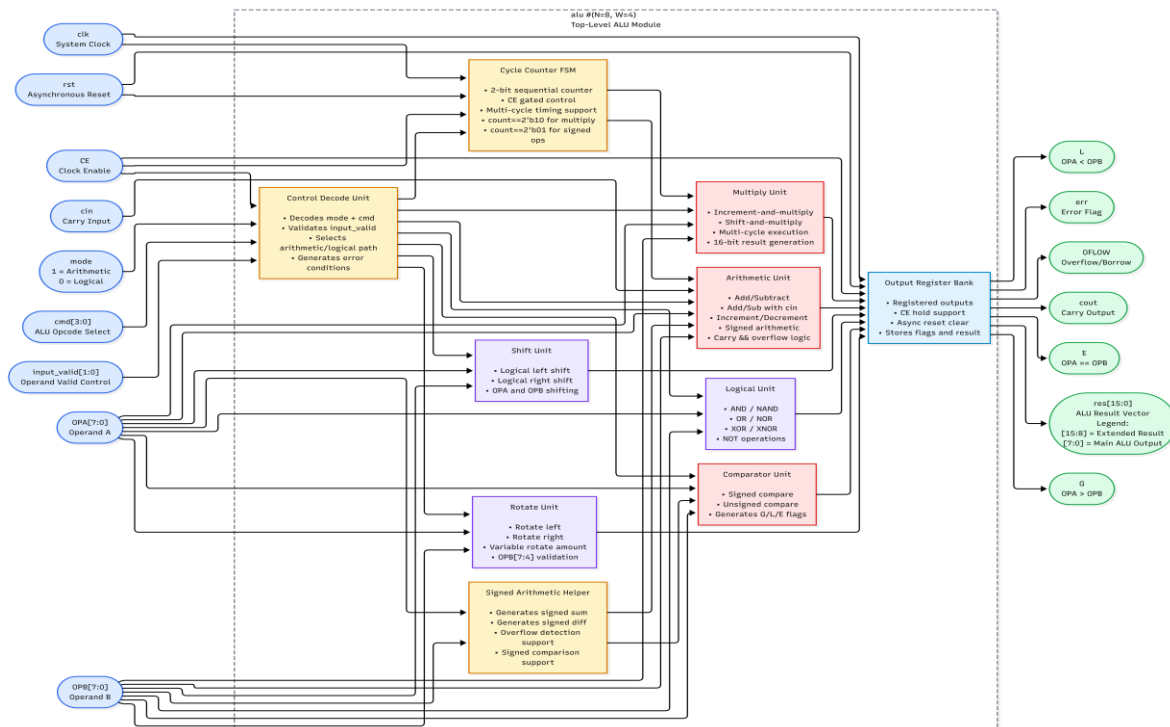
Arithmetic Operations (MODE = 1)

- CMD 0: Addition (ADD)
- CMD 1: Subtraction (SUB)
- CMD 2: Addition with Carry-in (ADD_CIN)
- CMD 3: Subtraction with Carry-in (SUB_CIN)
- CMD 4: Increment Operand A (INC_A)
- CMD 5: Decrement Operand A (DEC_A)
- CMD 6: Increment Operand B (INC_B)
- CMD 7: Decrement Operand B (DEC_B)
- CMD 8: Compare (CMP)
- CMD 9: Increment Multiplication (INC_MULTIPLICATION) (Multi-cycle)
- CMD 10: Shift Operand A Multiplication (SHIFT_MULTIPLICATION) (Multi-cycle)
- CMD 11: Signed Addition
- CMD 12: Signed Subtraction

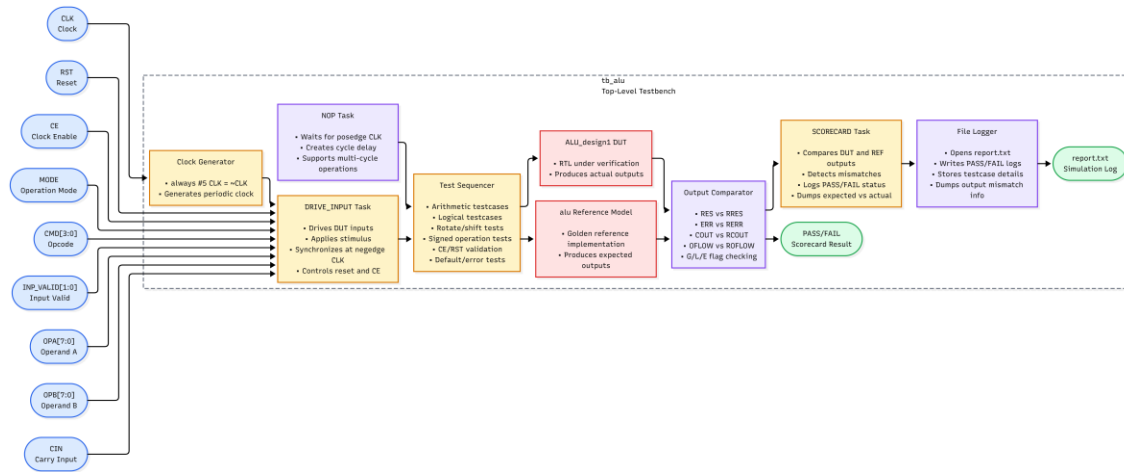
5. SYSTEM BLOCK DIAGRAMS

5.1 ALU Architecture Block Diagram

Based on the RTL design, the ALU utilizes an input-change detection system and a finite state machine to manage pipelined/multi-cycle operations before decoding the command.



5.2 Testbench Architecture Block Diagram



6. TIMING BEHAVIOUR

The design incorporates proper timing synchronization to ensure data integrity across the pipeline. Delay analysis was performed to ensure the ALU meets the required frequency constraints, with specific focus on multi-cycle operations such as pipelined multiplication.

- MODE 1 - > CMD(0,1,2,3,4,5,6,7,8,11,12) =>1 CYCLE DELAY
- MODE 0 - > CMD(0,1,2,3,4,5,6,7,8,9,10,11,12) =>1CYCLE DELAY
- MODE 1 - > CMD(9,10) =>2 CYCLE DELAY

7. TESTBENCH COMPONENTS

A robust, self-checking verification environment was implemented:

- Driver: Drives randomized and directed stimulus into the DUT.
- Monitor: Observes the DUT interface to capture transaction data.
- Reference Model: A behavioral model used to predict the expected result.
- Scoreboard: Compares the DUT output against the Reference Model to flag mismatches.
- All the result of test cases is being saved in report.txt.

8. QUALITY ASSESSMENT & RESULTS

Verification was carried out using QuestaSim. During the assessment, the following issues were identified:

BUGS IDENTIFIED :

- INPUT_VALID is mismatched for A and B .
- MULTIPLICATION output is taking extra one clock delay
- In SIGNED ADDITION the RES is holding previous value and G,L,E flags are not working if we gave input for SIGNED ADDITION after CMD 10.
- In LOGICAL OPERATION -> CMD 1, CMD 2, CMD 3, CMD 4 operations are not correct. Operations are not matching with DESIGN SPECIFICATION
- IN MODE 0 CMD = 6 ,7,9,10,11 INPUT_VALID are mismatched (10 for A and 01 for B)
- IN MODE 0 CMD 8 if INPUT_VALID!= 11 the error flag should be high.

LINT RESULT:

The screenshot displays the QuestaSim interface. On the left, the Project Manager and Flow Navigator are visible. The main window shows the Source File Properties for 'alu.v' and the Lint tool output. The Lint tool output shows 'No Violations found.'

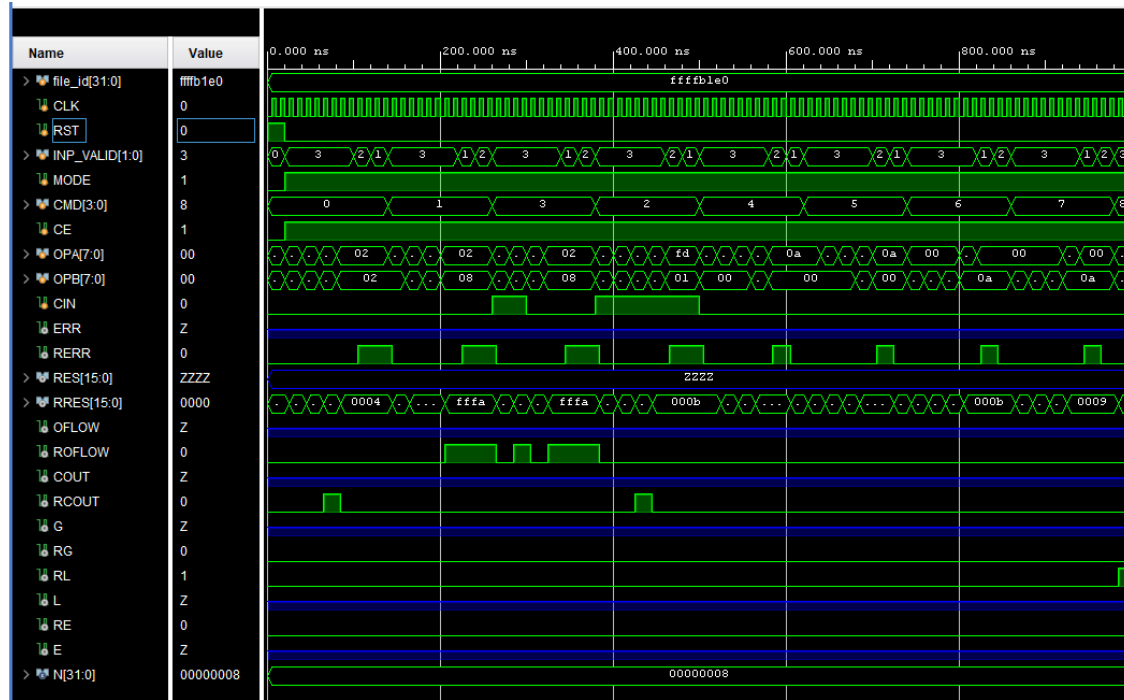
```
timescale 1ns / 1ps
module alu #(parameter N=8, parameter W=4)
(
    input wire clk,
    input wire rst,
    input wire [1:0] input_valid,
    input wire mode,
    input wire [N-1:0] cmd,
    input wire CE,
    input wire [N-1:0] OPA,
    input wire [N-1:0] OPB,
    input wire cin,

    output reg err,
    output reg [2*W-1:0] res,
    output reg OFLAG,
    output reg cout,
    output reg G,
    output reg L,
    output reg E
);
    reg [1:0] count;

    wire [N-1:0] diff = signed(OPA) - signed(OPB);
    wire [N-1:0] sum = signed(OPA) + signed(OPB);

    always @(posedge clk or posedge rst)
    begin
        if(rst)
            count <= 0;
    end
endmodule
```

WAVEFORM :



9. COVERAGE REPORT

Coverage analysis ensured that all RTL paths and functional scenarios were exercised. Coverage metrics include Statement, Branch, and functional coverage. The verification achieved high coverage targets, validating normal, corner, and error scenarios.

Centos

Wiki

Documentation

Forums

Testplan

Design

DesUnits

tb_alu

DRIVE_INPUT

nop

SCORECARD

dut

uut

Questa Design Coverage

Scope: [/tb_alu/dut](#)

Instance Path:
/tb_alu/dut

Design Unit Name:
[work.ALU_design1](#)

Language:
Verilog

Source File:
ALU_tb.v

Local Instance Coverage Details:

Total Coverage: 96.61% 95.42%

Coverage Type	Bins	Hits	Misses	Weight	% Hit	Coverage
Statements	91	86	5	1	94.50%	94.50%
Branches	51	46	5	1	90.19%	90.19%
FEC Expressions	7	7	0	1	100.00%	100.00%
FEC Conditions	15	14	1	1	93.33%	93.33%
Toggles	220	218	2	1	99.09%	99.09%

Coverage Type	Bins	Hits	Misses	% Hit	Observation
Statements	91	176	5	94.50%	Most code lines were executed, with only 5 lines (likely specific error cases) left unvisited.
Branches	51	125	5	90.19%	High coverage across if-else and case paths, missing only 5 specific logic routes.
FEC Expressions	7	4	0	100.00%	All logical expressions (e.g., A & B) have been tested to see how inputs affect the output.
FEC Conditions	15	24	1	93.00%	Nearly all individual logic conditions were tested independently, with only 1 miss.
Toggles	220	218	2	99.09%	Excellent signal activity; almost every bit flipped between 0 and 1 during the simulation.

10. CONCLUSION & FUTURE WORK

The project successfully demonstrated a fully functional, scalable ALU design. The verification environment proved effective in catching RTL bugs and ensuring timing synchronization. Future work will involve extending the ALU to support floating-point operations and further optimizing the power consumption of the arithmetic units and also adding division operation.